PRACTICAL ASSESSMENT · SENIOR .NET ENGINEER

BlockCypher Snapshot API

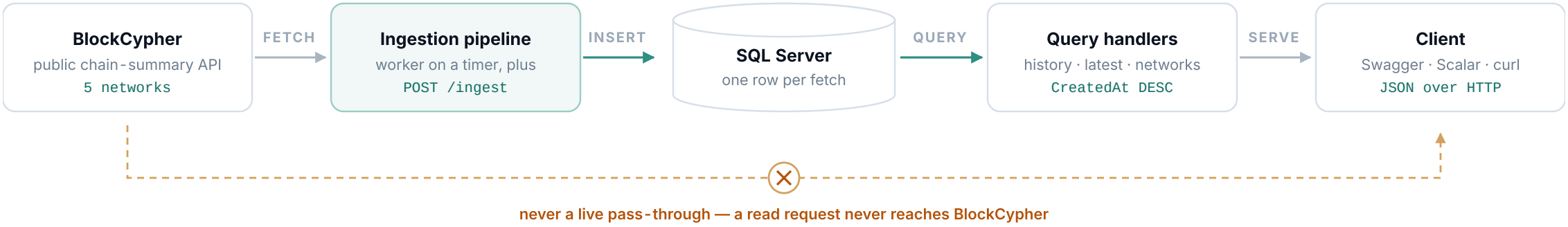
A .NET 10 Web API that ingests valid summaries for five blockchains, stores each snapshot with its request-start time, and serves the accumulated history newest-first.

RUNTIME	ARCHITECTURE	STORAGE	RUN PATH
.NET 10 (LTS) · C# 14	Clean + Vertical Slice + CQRS	SQL Server 2022 · EF Core 10	docker compose up --build

Updated 1 November 2026 · branch deveop2 · commit 3d0b2c1809a0

THE SOLUTION IN ONE PICTURE

Collect on a timer, store immutably, serve from the database

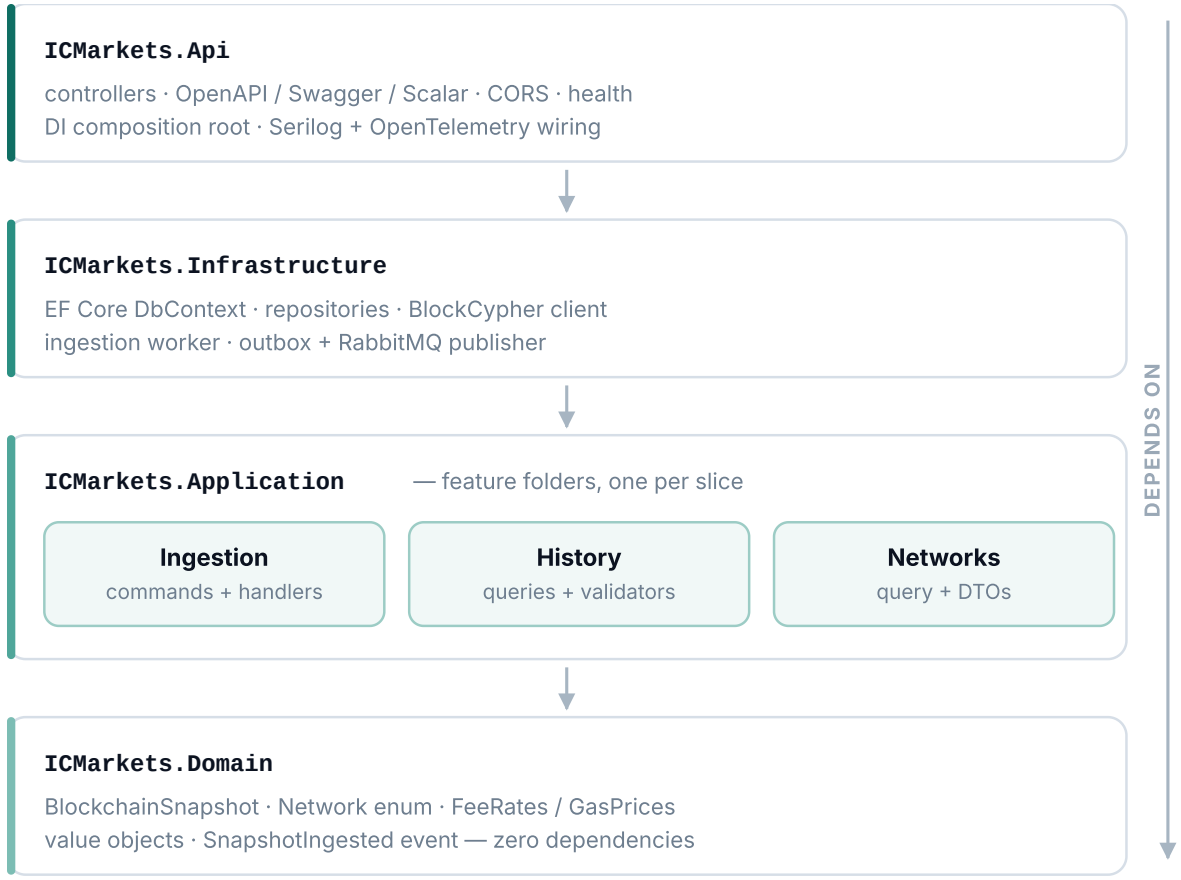


ENDPOINT	BEHAVIOUR
`POST /api/v1/blockchains/ingest`	Fetches all five networks. Completed batch: 200 with succeeded and failed; each network commits independently.
`GET …/{network}/history`	Keyset page: take (1–200, default 50) + cursor; items and nextCursor. Invalid input: 400.
`GET …/{network}/latest`	Most recent stored snapshot; `404` before the first ingestion.
`GET /health · /metrics`	Liveness excludes DB; readiness and /health probe DB. /metrics exposes the Prometheus scrape surface.

History contract. Pass nextCursor back as cursor; on the final or empty page, nextCursor is present as null. Null properties are retained for OpenAPI required + nullable fields: BTC gasPrices: null; ETH feeRates: null. Reads use stored data or the local network list.

ARCHITECTURE

Clean layers, with vertical feature slices inside Application



THREE PATTERNS, NAMED ON PURPOSE

Repository — `IBlockchainRepository` stages changes and never calls `SaveChanges` itself.

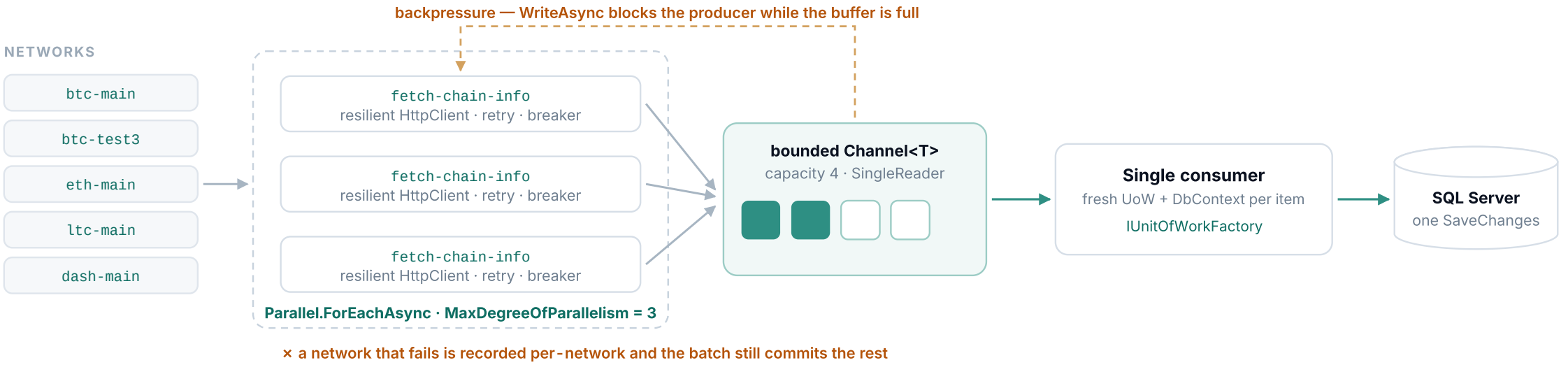
Unit of Work — IUnitOfWorkFactory isolates each snapshot. AppDbContext stages outbox rows through both async save overloads.

CQRS — MediatR commands/queries per feature. **V**alidators run sequentially, each with a fresh **V**alidationContext; controllers stay thin.

The boundary is explicit. IChainInfoSource returns neutral ChainInfo; Infrastructure owns the BlockCypher wire contract. IUnitOfWorkFactory hides DI scopes. Database:Provider selects SqlServer or Sqlite without changing Application or Domain.

Architecture tests guard Application against provider-named types, wire-format attributes, Infrastructure references and service-location signatures.

A bounded producer/consumer pipeline, not a fan-out



- **Thread safety by construction.** One consumer persists serially; every item gets its own unit of work, so a failed save cannot poison the next item.
- **Cancellation end to end.** Only caller/host cancellation aborts the batch. Fetch timeouts stay per-network; the writer completes in finally.
- **Request-start timestamp.** TimeProvider stamps RequestedAt before the source call; CreatedAt keeps it through retries, queueing and persistence.
- **Resilience at the edge.** Retry-After dates, broken-body retries and a shared 3-attempt/s limiter; default fetch budget: 45 s.

PLINQ was rejected deliberately. HTTP fetches are I/O-bound. Async producers are capped at 3; the bounded channel holds 4 items. A singleton rate gate sits inside the retry pipeline, so every actual HTTP attempt shares the process-wide quota.

Validate before storing. Reject empty/error payloads, foreign networks, implausible time and the wrong pricing family. Domain value objects also reject negative fees/gas; the factory guards counters and fork height.

DATA MODEL & THE HISTORY QUERY

Immutable snapshots, deterministic ordering, an index that matches

BlockchainSnapshot

Id
Network · nvarchar(450), index key
Height · Hash · Time
PeerCount · UnconfirmedCount · forks
RawJson · the response as provided
CreatedAt · UTC request-start time

`one row per fetch, never updated`

owns exactly one of

FeeRates
high / medium / low fee per kb
`BTC · LTC · Dash`

mutually exclusive

GasPrices
high / medium / low gas price
`ETH`

```
await context.Snapshots.AsNoTracking()
    .Where(s => s.Network == network)
    .OrderByDescending(s => s.CreatedAt)
    .ThenByDescending(s => s.Id)   // deterministic
.Take(take + 1).ToListAsync(ct);
```

Before ordering, a cursor adds: CreatedAt < cursor.CreatedAt OR equal time with Id < cursor.Id. The extra row detects another page; no OFFSET is used.

Two different times. Time comes from the upstream block. CreatedAt is captured immediately before the source call, before rate-limit waits, retries, response receipt or persistence. RawJson preserves the accepted response.

ORDERING, DONE PROPERLY

History orders by **CreatedAt DESC, Id DESC**, backed by (Network, CreatedAt DESC). The cursor is base64url of UTC ticks and Id. Newer inserts cannot shift the traversed boundary. It is parse-validated, unsigned, and used only for pagination.

READS

AsNoTracking with server-side keyset paging. The full entity is still loaded, including RawJson, then Mapperly drops it from the DTO. SQL Server uses EnableRetryOnFailure.

The provider difference is handled explicitly. SQLite stores CreatedAt and outbox ProcessedAt as UTC ticks so SQL comparisons translate; SQL Server keeps datetimeoffset. The design-time factory pins migrations to SQL Server. SQLite uses EnsureCreated.

TECHNOLOGIES, BY THE CONCERN THEY SERVE

Every dependency earns its place

PLATFORM

.NET 10 (LTS), C# 14, ASP.NET Core Web API with controllers. Central Package Management pins every version in one file.

PERSISTENCE

EF Core 10 + SQL Server 2022 (2022-latest tag); startup migrations. SQLite remains the configuration-selected local provider.

API SURFACE

Native **OpenAPI 3.1** (`Microsoft.AspNetCore.OpenApi`) with both **Swagger UI** and **Scalar** on top; RFC 7807 ProblemDetails.

APPLICATION PATTERNS

MediatR 12.5.0 for CQRS, **FluentValidation** via a pipeline behaviour, **Mapperly** source-generated mapping — no runtime reflection.

CONCURRENCY & RESILIENCE

Bounded Channels + Parallel.ForEachAsync; resilient HTTP with in-policy body buffering, shared attempt limiting and injected TimeProvider.

MESSAGING

Atomic snapshot + outbox save; RabbitMQ fanout with confirms and mandatory routing. At-least-once delivery; stable MessageId for deduplication.

OBSERVABILITY

OpenTelemetry traces and metrics; Prometheus + Grafana. Serilog separately emits ECS JSON to stdout, correlated with active trace/span IDs.

TESTING

xUnit, NSubstitute, AwesomeAssertions; Testcontainers.MsSql + RabbitMq; WireMock.Net, WebApplicationFactory and BenchmarkDotNet.

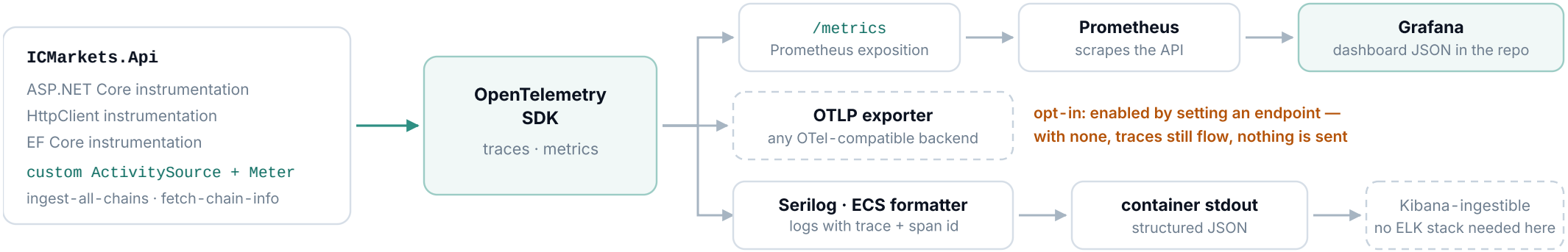
RUNTIME & DELIVERY

Multi-stage **Docker** images, **Docker Compose** as the verified path, **YARP** gateway, **Kubernetes** manifests, **GitHub Actions** CI.

Dependency choices are recorded in ADR 0003. Mapperly, pinned MediatR 12.5.0, AwesomeAssertions, NSubstitute and BenchmarkDotNet follow the repository policy. Central Package Management also enforces transitive pins, including System.Security.Cryptography.Pkcs 10.0.14.

OBSERVABILITY

Domain telemetry, not just framework defaults

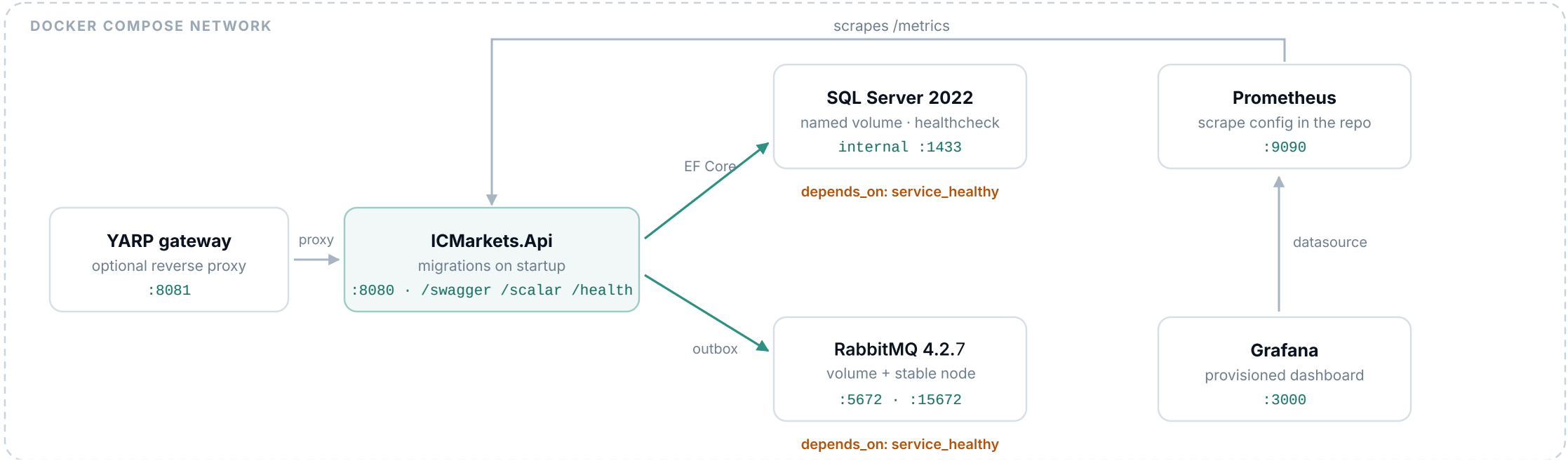


- **Logs correlate to traces.** Every line carries the active trace and span id, so a log entry leads back to the trace that produced it.
- **Domain metrics, not just framework counters** — snapshots per network, fetch-duration histogram, failure counter.
- **Separate probes.** /health/live excludes the DB; /health/ready and /health include database connectivity. RabbitMQ is not health-checked.
- **Nothing to configure** — the dashboard is provisioned in the repository and fills in on the first ingestion.

Why this is a core deliverable and not a bonus. OpenTelemetry appears in both halves of the role description. It is also the highest-leverage addition to an otherwise ordinary CRUD assessment: a reviewer who runs one command and watches ingestion metrics move remembers the submission.

RUNTIME TOPOLOGY

One command brings up the whole stack



Startup validation. IValidateOptions + ValidateOnStart check BlockCypher URL (absolute HTTP(S)), all network paths and positive fetch/rate settings; the ingestion interval must be positive. When outbox is enabled: a positive number of rows per batch, positive poll/publish intervals and retention, non-empty host/exchange, port 1–65535. Invalid configuration stops startup; validation makes no upstream call.

Durable outbound delivery. Snapshot + event commit together. Publish confirms and mandatory routing precede ProcessedAt. A 10 s publish budget prevents a stuck tick; failures retry with the same Id. Consumers deduplicate; processed rows expire after 24 h.

Deployment boundary. Compose persists SQL and broker data. k8s/ is untested: RabbitMQ has a PVC; SQL Server does not. Local appsettings selects SQLite with outbox disabled. Scale-out needs global quotas and coordination of ingestion/outbox workers.

TESTING

Four projects, each testing something the others cannot

69
Unit test methods — invariants, handlers, ports, validators and mapping

31
Integration methods — SQL Server, RabbitMQ, outbox and worker resilience

32
Functional methods — full HTTP pipeline, isolated upstream and save failures

2
BenchmarkDotNet suites on the mapping and query-handler paths

WHAT EACH SUITE EXERCISES	Unit	Integration	Functional	Performance
HTTP pipeline, controllers, DI	—	—	real	—
Command / query handlers	real	—	real	real
Repository + EF Core	in-memory fake	real	real	in-memory fake
Database engine	—	SQL Server / SQLite	SQLite in-memory	—
BlockCypher upstream	NSubstitute	—	WireMock.Net	—

the real component runs · substituted on purpose · not exercised by that suite

Source method counts; suites and benchmarks not rerun for this PDF.

Regression coverage. Request timestamps, cursors, pricing invariants, broken/stalled bodies, shared limits, Retry-After dates, fresh contexts and atomic outbox staging. New tests check required JSON fields (including null) and reject invalid startup options.

CI is an execution gate. Pushes to every branch; PRs to main/development. Release build uses warnings-as-errors. REQUIRE_SQLSERVER=1 and REQUIRE_RABBITMQ=1 make missing containers fail CI; local container-dependent tests explicitly skip.

DECISIONS WORTH DEFENDING

Where the obvious answer was the wrong one

DECISION	REASONING
.NET 10, not .NET 8	The checked-in target is .NET 10 / C# 14; global.json selects SDK 10.0.401 with latestMinor roll-forward. ADR 0001 records the runtime decision. Docker remains the primary documented run path.
SQL Server 2022 — with a SQLite fallback	SQL Server is the compose provider; SQLite is the local/functional-test provider. 2022-latest fixes the SQL Server major, but remains a mutable image tag. SQL Server keeps datetimeoffset and migrations; SQLite uses UTC-tick conversions and EnsureCreated.
Channels + bounded parallelism, not PLINQ	HTTP fetches are I/O-bound. Async concurrency is capped at 3 with channel capacity 4. A separate singleton limiter spaces every real HTTP attempt, including retries; each network persists in a fresh unit of work.
Permissive dependencies only	The repository policy selects Mapperly, MediatR 12.5.0, AwesomeAssertions, NSubstitute and BenchmarkDotNet. Central versions and transitive pinning keep these decisions explicit. See ADR 0003 for the recorded dependency-selection rationale.
DDD vocabulary, scoped short	One snapshot aggregate, two pricing value objects and a domain event. The factory enforces network-specific fee/gas families and non-negative values; the adapter validates upstream shape. This is one snapshot table, not five aggregates.
Stored history, never a live pass-through	Reads use the database. CreatedAt means request start: captured before the source call and retained through rate-limit waits, retries and the channel. It is distinct from upstream block Time and from persistence time.

SCOPE, HONESTLY

What is deliberately out, and where it would go next

BEYOND THE BRIEF, ALREADY IN

OpenTelemetry traces/metrics, Prometheus and a Grafana dashboard

Transactional outbox and RabbitMQ event publishing

YARP API gateway and Kubernetes manifests

GitHub Actions CI and BenchmarkDotNet performance tests

Five ADRs plus a decision log covering every non-obvious choice

Retention is explicit. Demo queue: 24 h TTL / 10,000 messages. Processed outbox rows expire after 24 h; pending events and snapshots remain.

REVIEWING IT IN FIVE MINUTES

1. `docker compose up --build`, then open `:8080/swagger`.
2. Call `POST /ingest` once, then `GET …/btc-main/history` — and watch the Grafana dashboard on `:3000` fill in.
3. `dotnet test` for the three suites; `docs/adr/` for the reasoning behind every choice above.

NOT IMPLEMENTED	WHY / NEXT STEP
Authentication	Out of the brief's scope. A real deployment sits behind JWT/OAuth2 with the CORS policy scoped to actual origins.
Distributed cache	`latest` hits the database per call; a short-TTL Redis read-through would cut load without touching ingestion correctness.
Kafka / SQS	`IEventPublisher` is the seam — a new implementation, not a redesign. RabbitMQ was the cheapest real broker to run in compose.
Scale-out coordination	Rate quotas and workers are per process; no outbox row lease exists. Add global quotas, scheduling and idempotent consumers before scaling.
Hardened docs exposure	Swagger and Scalar are reachable in every environment so a reviewer needs no environment switch; production would gate them behind a flag.

Read-path trade-offs. No cache or server-side DTO projection; reads still load RawJson. Kubernetes manifests are not cluster-tested. Forwarded headers are opt-in and restricted to configured proxies/networks.